AutoVet

Security and Audit Log Update

A simple guide for the capstone defense: what was changed, why, how it was tested, and how to explain it to the panel.

Item	Details
Date of changes	November 7, 2026
Branch	Seb1 (deploys automatically to Render and Vercel)
Commits	15ffbd6e - sign-up protection a4e8a279 - audit log upgrade 4278a52b - audit log role labels
Live system	Admin: autovet-admin.vercel.app | Backend: autovet.onrender.com
Tests	83 automated backend tests passed after the audit log upgrade, plus 1 new test for the role fix (the audit log tests: 9 of 9 passed)

Contents

- 1. Summary of the three changes
- 2. Change 1 - Protection against fake sign-ups
- 3. Change 2 - More accurate audit logs
- 4. Change 3 - Correct role labels in the audit log
- 5. Where to find it in the source code (for code walkthroughs)
- 6. Question-and-answer preparation (Part 3)
- 7. System readiness checklist (Part 2)
- 8. How this supports the evaluation criteria (Part 5)
- 9. Known limitations (be honest with the panel)
- 10. Forecast model: the linear regression equation

1. Summary of the three changes

Change	Problem before	What we did
1. Sign-up protection	Anyone could send many sign-up requests. Because all visitors reach the server through the same proxy, one person could also use up the limit for everybody.	Limits now apply to each visitor separately. Sign-ups are capped per minute, per day and per email address. An optional "I am human" check (Cloudflare Turnstile) was added.
2. Audit log upgrade	The log saved password hashes, saved full copies of records on every edit, missed sign-ins and archives, and some filters did not work.	Secrets are hidden, only changed fields are saved, more events are recorded, search and filters were added, and old entries are cleaned up automatically.
3. Role labels	A clinic admin was shown as "Staff" in the "Done by" filter.	The page now shows and filters by the person's real role (Clinic Admin, Veterinarian, Staff, Super Admin).

One-sentence version for the panel: We made it harder to abuse the sign-up page, and we made the audit log record who did what more accurately, without storing secrets and without letting it grow too large.

2. Change 1 - Protection against fake sign-ups

What is a rate limit?

A rate limit is a rule like "you may only try this 5 times per minute". After that, the server answers "Too many attempts" (HTTP 429) until the time window passes. It slows down robots that try to create many accounts or send many emails.

What was wrong

- **Shared limit.** The website (Vercel) forwards every request to the backend (Render). The backend only saw the proxy's address, so every visitor looked like the same person. One abuser could block sign-up and password reset for everyone.
- **Email spam.** Each sign-up sends a verification email. Someone could type other people's email addresses and make the system send them many emails, using up our email quota.
- **No robot check** on the sign-up form.

Note: fake accounts were already partly blocked, because an account is only created after the person clicks the link in the verification email. The bigger risks were the email spam and the shared lockout.

What we did

Protection	Rule
Per visitor	Maximum 5 sign-up attempts per minute
Per visitor, per day	Maximum 20 sign-up attempts per day
Per email address	Maximum 5 attempts per hour (from anywhere), so one inbox cannot be flooded
Real visitor address	Limits use the visitor's real IP address taken from the X-Forwarded-For header, which Vercel sets. This is used only for rate limits.
Robot check (optional)	Cloudflare Turnstile on the portal sign-up form. It is OFF until two keys are set: TURNSTILE_SECRET_KEY (Render) and VITE_TURNSTILE_SITE_KEY (Vercel).

The same "real visitor address" fix was also applied to the login, password reset and Google sign-in limits.

How it was tested

- Automated test: one email address gets blocked after 5 attempts in an hour.
- Automated test: when one visitor is blocked, a different visitor can still sign up.
- Automated test: when Turnstile is turned on, a fake token is rejected.

Important: If you set only one of the two Turnstile keys, every sign-up will be refused. Set both or neither. If unsure before the defense, leave both empty.

3. Change 2 - More accurate audit logs

What is an audit log?

An audit log is a history of important actions: who did what, to which record, and when. Example: "Dr. Reyes changed pet Bella's weight from 4 kg to 5 kg on Nov 7 at 2:15 PM." Admins can view it in **Settings > Audit Logs**.

Problems we found

- **Password hashes were being saved in the log.** When a staff or client account was created or edited, the whole record (including the scrambled password) was copied into the log and could be viewed by admins. On the local database, 8 of 23 log entries had one.
- Every edit saved two full copies of the record, even if only one field changed. This wastes storage.
- Client actions showed as "System" instead of the client's name.
- The IP address saved was the proxy's address, not the real visitor's.
- Sign-ins, sign-outs, failed sign-ins, archiving and restoring were not recorded.
- Deleting a staff account also deleted all of that person's history in the log.
- The "Model / Target" filter never worked (typing "Pet" found nothing).

What we fixed

Goal	What the system does now
Security	Passwords, tokens and other secret fields are saved as "[hidden]". The page shows "Password: changed (value hidden)". Old entries were cleaned by a database migration and are filtered again when read.
Accuracy	Each entry saves the person's name and role at that moment, whether they are staff, a client, a guest (not signed in) or the system, the record's name (for example the pet's name), the real IP address, and a request ID that groups all changes made in one save.
More events	Signed in, signed out, failed sign-in, archived, restored, password reset, and super admin impersonation (shown as "via super admin").
History is kept	Deleting a staff account no longer deletes their log entries.
Easy tracking	Search box (record or person name), "Done by" filter, more action types, a working record-type filter, and an "Everything in this action" link.

Keeping storage and memory small

- Edits save **only the fields that changed**, not the whole record twice.
- Saves that only change the time stamp are not logged at all.
- Long text is cut to 500 characters. Attached data (like JSON blobs) is not copied.

- Failed sign-ins are logged at most once per account every 10 minutes, so password guessing cannot flood the log.
- Entries older than 365 days are deleted automatically, 500 at a time, while new entries are written. No scheduled job is needed. The number of days can be changed with AUDIT_RETENTION_DAYS (0 = keep forever).
- The page loads at most 50 entries per page and does not download unused data.

No new endpoints

No new web address (API route) was added. The existing **/api/audit-logs** page was improved. It still requires a signed-in clinic admin, veterinarian or super admin, and each clinic only sees its own entries.

How it was tested

- Edits save only the changed fields and the correct person.
- Time-stamp-only saves create no entry.
- Password hashes never reach the audit table, and old entries with secrets are hidden when read.
- Long text is shortened.
- Sign-ins are recorded; 3 wrong passwords in a row create only 1 failed sign-in entry.
- Typing "Admin" in the record-type filter works.
- Entries older than the retention period are deleted.
- The "Done by" filter matches the role (see Change 3).

4. Change 3 - Correct role labels in the audit log

Problem: A clinic admin was shown as "Staff" in the audit log. Inside the system, every clinic account (clinic admin, veterinarian, staff) is stored in the same accounts table. The audit log labeled that group "staff", which is also the name of one real role, so it looked wrong.

Fix: The "Done by" filter now lists Clinic Admin, Veterinarian, Staff, Super Admin, Client, Super Admin (impersonating), Not signed in, and System. It uses the role saved with each entry. The person's role is also shown under their name in the table and in the details window.

5. Where to find it in the source code

Use this table if the panel asks you to show and explain the code. All paths are inside the **backend** folder unless stated.

File	What it does
`routes/api.php (line 63)`	Connects the sign-up route to the new "register" rate limit.
`app/Providers/AppServiceProvider.php (line 111)`	Defines the sign-up limits: 5 per minute, 5 per hour per email, 20 per day.
`app/Support/ClientIp.php`	Finds the visitor's real IP address for rate limits.
`app/Support/Turnstile.php`	Asks Cloudflare whether the robot check was passed. Does nothing if no key is set.
`app/Http/Controllers/AuthController.php (line 54)`	Sign-up checks Turnstile after validating the form.
`app/Support/AuditLogger.php`	The one place where audit entries are written. Key parts: record() writes an entry, diff() keeps only changed fields, redact() hides secrets and shortens text, prune() deletes old entries.
`app/Traits/HasAuditTrail.php`	Automatically logs create, update, archive, delete and restore for models like Pet, Owner, Appointment, Invoice and Admin.
`app/Models/AuditLog.php`	Hides secrets again whenever an entry is read.
`app/Http/Controllers/AuditLogController.php`	The audit log page's data: filters, search, clinic restriction.
`database/migrations/2026_11_07_000001_improve_audit_logs.php`	Adds the new columns and removes old password hashes from existing entries.
`frontend/admin/src/components/settings/AuditLogTab.jsx`	The audit log screen in the admin app.
`frontend/portal/src/components/Turnstile.tsx`	The robot check box on the portal sign-up page.
`tests/Feature/RegistrationAbuseTest.php` `tests/Feature/AuditTrailTest.php`	Automated tests for both features.

Simple way to explain the audit logger

> "Every important model uses a trait called HasAuditTrail. When a record is created, updated or deleted, Laravel fires an event. The trait catches that event and calls AuditLogger::record(). Before saving, the logger removes secrets, keeps only the changed fields, and adds who did it, their role, the IP address and a request ID."

6. Question-and-answer preparation (Part 3)

Possible panel questions about these changes, with short answers. Every group member should be able to answer these.

Security and Data Management

Q: How do you stop people from creating many fake accounts?

A: An account is only created after the email link is clicked. On top of that, sign-up has rate limits (5 per minute and 20 per day per visitor, 5 per hour per email) and an optional Cloudflare Turnstile robot check.

Q: What is a rate limit and what happens when it is reached?

A: A maximum number of attempts in a time window. When reached, the server replies with HTTP 429 "Too Many Requests" and the user must wait.

Q: Why did you need the visitor's real IP address?

A: Requests pass through Vercel to Render, so the server only saw the proxy's address. Without the real address, everyone shared one limit, and one person could block everyone.

Q: Can the IP address be faked?

A: Through the normal website, no, because Vercel overwrites the header. Someone calling the backend directly could fake it, but that only gives them a new rate-limit counter. That is why the robot check matters. The value is used for rate limits only, not for anything else.

Q: How do you protect passwords?

A: Passwords are stored as one-way hashes (bcrypt) in the database. After this update they are also never copied into the audit log. Secret fields are saved as "[hidden]".

Q: Who can see the audit logs?

A: Only signed-in clinic admins, veterinarians and super admins. Each clinic sees only its own entries. A super admin can see all clinics.

Q: How do you stop the audit log from filling the database?

A: Only changed fields are saved, long text is cut to 500 characters, failed sign-ins are limited to one entry per 10 minutes per account, and entries older than 365 days are deleted automatically.

Q: What about backups?

A: The system has a backup command, php artisan db:backup, that exports key clinic data as CSV files in a compressed file. (Note: the live server on Render does not run scheduled jobs, so run the backup manually before the defense.)

Technical Implementation

Q: What technologies are used?

A: Backend: Laravel (PHP) on Render. Database: TiDB Cloud (MySQL-compatible). Frontends: React apps on Vercel. Robot check: Cloudflare Turnstile. Email: Brevo.

Q: How does the audit log record changes automatically?

A: Through Laravel model events. The HasAuditTrail trait listens for created, updated, deleted and restored events and calls AuditLogger::record().

Q: What is the request ID for?

A: One save can change several records (for example an invoice and its items). They all get the same request ID, so the "Everything in this action" link can show them together.

Q: How did you test these changes?

A: With automated PHPUnit feature tests that run against a separate test database. The full backend suite of 83 tests passed after the audit log upgrade.

System Functionality

Q: What events does the audit log record?

A: Created, updated, archived, deleted, restored, signed in, signed out, failed sign-in, password changes, client account suspension, deactivation and reactivation, and super admin impersonation.

Q: How does an admin find a specific activity?

A: In Settings > Audit Logs, use the search box (record or person name), the User, Done by, Action, record type and date filters, or click "Everything in this action".

Q: Why was the clinic admin shown as Staff before?

A: All clinic accounts are in one table, and the log labeled that group "staff". We changed the filter and labels to use each person's real role.

7. System readiness checklist (Part 2)

Tick each item before the defense day.

	Task	How
[ ]	Live system is awake	Open autovet.onrender.com/up one hour before. The free Render server sleeps after 15 minutes without visits (an UptimeRobot pinger keeps it awake).
[ ]	Admin app works	Sign in at autovet-admin.vercel.app as a clinic admin.
[ ]	Audit log works	Open Settings > Audit Logs. Your sign-in should appear. Edit a pet, then check that only the changed field is shown.
[ ]	Role filter works	Choose "Clinic Admin" in "Done by" and confirm your entries appear.
[ ]	Sign-up works	Create a test account on the portal and click the email link.
[ ]	Turnstile decision	Either set BOTH keys and test sign-up, or leave BOTH empty.
[ ]	Internet	Test the venue Wi-Fi. Prepare a mobile hotspot as backup.
[ ]	Database backup	Run php artisan db:backup and copy the file to a USB drive and a cloud drive.
[ ]	Source code backup	Make sure all work is pushed to GitHub (branch Seb1). Keep a ZIP copy offline.
[ ]	Local copy (Plan B)	Make sure the system runs on a laptop with Laragon (MySQL started) in case the internet or Render fails.
[ ]	Demo accounts	Prepare and test the accounts for each role you will show.
[ ]	Documents	Bring printed and digital copies of the manuscript and these guides.

Contingency plan (if something goes wrong)

Problem	What to do
Live site is slow on first load	Render is waking up. Wait about 1 minute and reload.
No internet at the venue	Use the mobile hotspot, or switch to the local Laragon copy.
Render or Vercel is down	Demonstrate using the local copy on the laptop.
Sign-up shows "Too many attempts"	The rate limit is working. Wait a minute (up to an hour if the same email was tried 5 times), or use a different prepared account. This is also a good live demo of the feature.
Sign-up always fails with a security check error	Only one Turnstile key is set. Set both or remove both.
Audit log is empty	Check the date filters and "Done by" are cleared, then do an action (sign in or edit) and reload.

8. How this supports the evaluation criteria (Part 5)

Panel criterion	How these changes help
System functionality and completeness	More activities are recorded (sign-ins, archives, restores). Filters that did not work now work.
System usability and user experience	Search box, plain-language action names, the person's real role, and "Everything in this action" make the log easier to read.

Panel criterion	How these changes help
Technical implementation and performance	One central logger, only changed fields saved, automatic clean-up of old entries, smaller page responses.
Security and data management	Rate limits per visitor and per email, optional robot check, no secrets in the log, history kept when accounts are deleted.
Compliance with project requirements	No new endpoints added; access to logs stays limited to admin roles and to each clinic's own data.
Quality and completeness of documentation	This guide, code comments in each file, and clear commit messages.
Proponents' knowledge	Use sections 5 and 6 to practise explaining the code and answering questions.

9. Known limitations (be honest with the panel)

- The robot check (Turnstile) is off until both keys are set.
- Someone calling the backend directly (not through the website) can fake their IP to get a new rate-limit counter. The robot check is the defense against this.
- Entries need a clinic. Super admin actions that do not belong to a clinic, and clients not linked to a clinic, are not logged.
- Changes made with bulk database queries (not through the model) do not trigger the audit log.
- Entries created before this update do not have the client's name or the record label, because that information was not saved at the time.
- Entries older than 365 days are deleted automatically, including old demo data. Set AUDIT_RETENTION_DAYS=0 to keep everything.
- Scheduled jobs (like automatic backups) do not run on the free Render plan; run backups manually.

Recommendation for future work: turn on Turnstile in production, add a scheduled backup that runs outside Render, and log super admin actions that are not tied to a clinic.

10. Forecast model: the linear regression equation

The inventory forecast uses **simple linear regression** (scikit-learn **LinearRegression**, one input variable). The code is in **backend/ai/forecast.py** (one item) and **backend/ai/batch_forecast.py** (many items at once). Both use the same equation.

Step 1 - Prepare the data

The model does **not** fit the stock level directly, because restocks make the stock level jump up and break a straight line. Instead it fits the **running total of units used**, which only goes up (forecast.py, lines 139-145):

- **Daily usage** = how much the stock dropped that day (a restock counts as 0).
- **Running total** = daily usage added up day by day.

Step 2 - The regression equation

$$\text{Predicted } y = b_0 + b_1 x$$

Symbol	Meaning in AutoVet
x	Day number, counted from the first day of the usage history (0, 1, 2, ...)
y	Running total of units used up to that day (predicted y = the value on the fitted line)
b_1 (slope)	**Units used per day.** This is the number the forecast actually uses.
b_0 (intercept)	Where the line starts. It is saved, but not used for the prediction.

Step 3 - How b_1 and b_0 are calculated (least squares)

$$b_1 = \Sigma(x - \text{mean } x)(y - \text{mean } y) \,/\, \Sigma(x - \text{mean } x)^2$$

$$b_0 = \text{mean } y - b_1 \cdot \text{mean } x$$

Σ means "add up for every day". This is the standard line of best fit: the line with the smallest total squared error.

Step 4 - Predict when the item reaches its minimum stock

$$\text{Days left} = (\text{Current stock} - \text{Minimum stock level}) \,/\, b_1$$

The result is rounded up to a whole day (forecast.py, lines 233-234).

$$\text{Expected date} = \text{Today} + \text{Days left}$$

Result	Status shown
Fewer than 7 days left	Critical
7 to 13 days left	Reorder Soon
14 days or more	Safe
Stock already at or below the minimum	Critical
b_1 is 0 or less (item not being used)	Safe - no stockout predicted

Step 5 - How well the line fits (R^2)

$$R^2 = 1 - \Sigma(y - \text{predicted } y)^2 / \Sigma(y - \text{mean } y)^2$$

R^2	Confidence shown
0.7 or higher	High
0.4 to below 0.7	Medium
Below 0.4	Low (the system shows a warning to collect more data)

When an item has at least 10 days of data, the model is also tested properly: it learns from the first 80% of days and is scored on the last 20% (test R^2, RMSE and MAE). The forecast shown to users is then made by a model trained on 100% of the data.

Worked example

A vaccine's running total of units used gives a slope of **b_1 = 2.5 units per day**. Current stock is **40** and the minimum stock level is **10**.

Days left = (40 - 10) / 2.5 = **12 days**, so the status is **Reorder Soon** (between 7 and 13 days).

One-sentence answer for the panel

"We fit a straight line, $y = b_0 + b_1x$, to the running total of units used over time. The slope b_1 is the daily usage rate, and we divide the stock left above the minimum by that rate to predict how many days until the item needs restocking."